

SiteComply — Project Handover

Repository state: latest commit c13eb6e (branch feature/archived-badge-style). **Audience:** a developer with no prior knowledge of the project who must continue development from this document alone. **Live production URL:** <https://app.sitecomply.co.uk>

Companion documents in-repo: docs/RBAC.md (permission model), docs/REPORTS.md (Reports design + phased plan), DEPLOYMENT.md, LOCALISATION.md, README.md.

1. Executive summary

SiteComply is a **digital site-induction and compliance check-in platform for UK construction**. It replaces paper sign-in sheets: a worker verifies their UK mobile by SMS one-time passcode, completes a short digital induction (site rules, PPE, safe working, UK-GDPR consent), and is recorded as **compliant and checked in**. Site administrators sign in with Microsoft (Azure AD) to manage sites, induction checklists, and view/export attendance and compliance records, with UK-GDPR personal-data erasure.

A second surface — the **Platform** area — is being built for office/management users ("Platform Users": Directors, Project Managers, Clients, Auditors, Engineers, H&S Consultants, Principal Contractors, Site Managers). It provides role-based, site-scoped oversight and reporting. **Platform Login, Assigned-Sites enforcement, and role-based permissions (RBAC) are complete and live**. The **Reports module** is in progress: **Phase 0 (foundation) is complete and committed**; Phase 1 (first reports) has not yet started.

The core worker + admin product is **live in production**. The Platform/RBAC layer is live. The Reports module foundation is committed but **not yet deployed**.

2. Business purpose

- **Problem:** paper site sign-in/induction is slow, unauditable, and hard to report on for compliance (CDM, H&S).
 - **Solution:** phone-based induction + check-in for workers; a web dashboard for site admins; and a management/reporting layer for office users.
 - **Users:**
 - **Workers** — trades on site; identified by UK mobile (SMS OTP). No account/password.
 - **Admins** — SiteComply staff / site operators; Microsoft SSO; full site + data control.
 - **Platform Users** — client organisation's office/management staff; role-scoped read/reporting access to their assigned sites.
 - **Compliance drivers:** UK GDPR / Data Protection Act 2018 (minimal personal data, erasable), CDM/H&S evidence (who was inducted, when, PPE/rules acknowledged).
-

3. Current development status

Area	Status
Worker check-in (SMS OTP → induction → compliant → check-out)	✓ Live in production
Admin dashboard (Microsoft SSO): sites, checklist builder, submissions, on-site, GDPR erase	✓ Live
Branding (logo, favicon set, custom domain + TLS)	✓ Live
Platform Users management (admin CRUD, roles, status, assigned sites)	✓ Live
Platform Login (email/mobile) with Active-only validation	✓ Live (dev verification code)
Assigned-Sites enforcement (Director all / others assigned)	✓ Live
RBAC — role-based permissions, all 8 roles (nav, page, buttons, exports, API)	✓ Live
Reports module Phase 0 (foundation, registry, audit log)	✓ Committed (c13eb6e) — not deployed
Reports Phase 1+ (actual reports)	✗ Not started
Documents / Audits / Actions modules	✗ UI preview stubs only
Real SMS/email sending	✗ Mock only (see §22)

Production is running commit b2a9f46 (RBAC Phase 2). Commits c13eb6e (Reports Phase 0) is pushed but **not deployed**, and its migration is **not yet applied to the production database**.

4. Completed phases & features (chronological)

1. **Core product (v1)** — worker SMS check-in, digital induction wizard, compliance capture, check-out, confirmation screen; admin dashboard, sites CRUD, versioned checklist builder, submissions list + detail + CSV export, "On site now", worker personal-data erasure (GDPR).
2. **Branding** — SiteComply logo, full favicon set (favicon.ico, apple-icon, PWA icons), custom domain `app.sitecomply.co.uk` with managed TLS.
3. **Site-management polish** — archive/restore with confirm dialog, permanent delete with check-in guard, archived-badge styling, toast notifications.
4. **Platform Users management** (admin) — add/view/edit/approve/disable/remove; fields: name, company,

email, mobile, role, status, assigned sites; PlatformUser model.

5. **Platform Login** — landing "Platform Login" → email or mobile → verification code → dashboard. Validates against PlatformUser (only ACTIVE may sign in; PENDING → "awaiting approval"; DISABLED → "access revoked"). **Development verification code 123456** (no real email/SMS yet).
 6. **Platform Dashboard + sections** — left-nav shell distinct from admin; site-scoped dashboard (Active Sites, Workers On Site, real check-in activity); section pages for Sites / Submissions / Reports / Documents / Audits / Actions.
 7. **Assigned-Sites enforcement** — signed platform session; Director sees all sites, every other role only their assigned sites; status re-checked per request.
 8. **RBAC (Phase 2, all roles)** — approved matrix (docs/RBAC.md) enforced via `permits()` across navigation, page access, action buttons, exports, and API routes; read-only Client; scoped CSV exports for Sites and Check-ins.
 9. **Reports module Phase 0** — design doc, report-type registry, access/scope helpers, shared CSV util, ReportExportLog audit model, role/scope-filtered Reports landing.
-

5. Current phase

Reports module — Phase 0 complete; Phase 1 next. Development was paused at Phase 0 (committed c13eb6e) to produce this handover. Phase 1 = **Site Attendance + Compliance reports** (scoped CSV export, Client aggregate-only, audit logging wired in). No Phase 1 code has been written.

6. Pending phases & features

Reports module (see docs/REPORTS.md):

- **Phase 1** — Site Attendance + Compliance (CSV, Client aggregate-only, logged).
- **Phase 2** — On-Site Occupancy + Workforce/Company.
- **Phase 3** — CSCS/Competency + **Expiring CSCS Cards (30/60/90-day)**; export restricted to Director/PM/Site Manager/H&S; add `worker.cscsExpiry` index.
- **Phase 4** — Director Organisation Overview (org-wide rollup).
- **Phase 5** — Hardening + **Admin-only** export-log viewer + tests.

Other modules (currently UI preview stubs — no data models/logic):

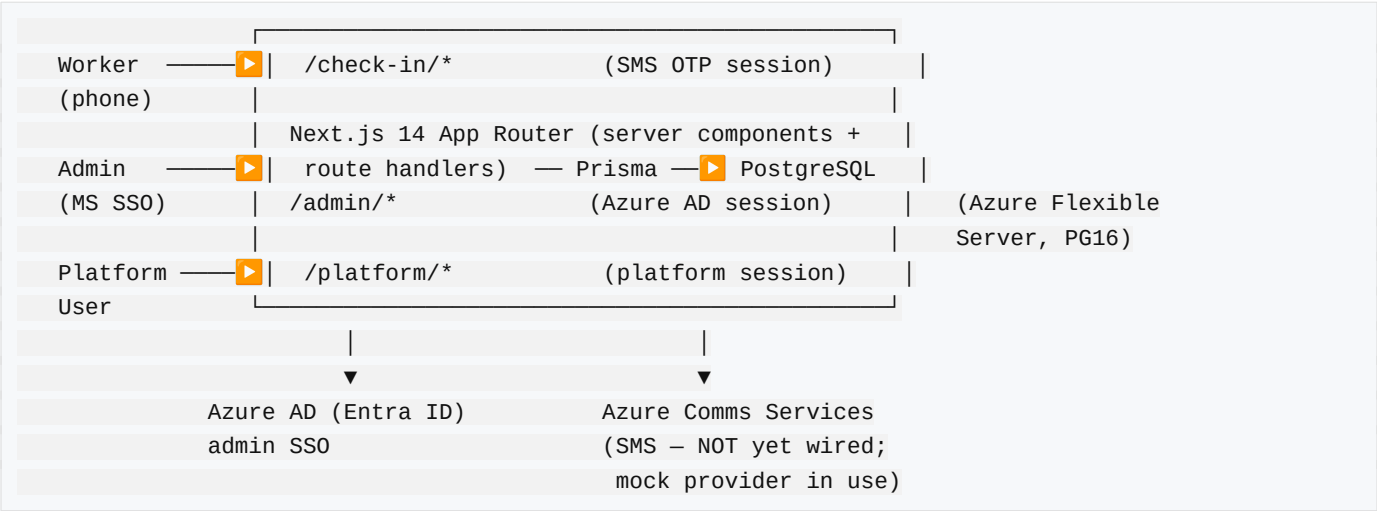
- **Documents** — RAMS / method statements / certificates library.
- **Audits** — site inspections + audit sign-off.
- **Actions** — tasks / follow-ups.

Platform / infra deferred:

- Replace the **dev verification code** with real email + SMS OTP for platform login.
- Real worker SMS (Azure Communication Services — provider not yet registered).
- Saved/scheduled reports, email delivery, custom report builder, PDF export.
- Key Vault for secrets; Application Insights; automated tests; CI/CD.

7. Architecture overview

Single **Next.js 14 (App Router)** application — server components + route handlers — backed by **PostgreSQL via Prisma**, deployed to **Azure App Service (Linux)**. There is no separate backend service; API route handlers under `app/api/**` are the backend. Three independent auth surfaces each use signed-cookie sessions.



Key cross-cutting layers

- **Sessions** — `lib/session.ts`: HMAC-signed base64url cookies (`sc_worker`, `sc_admin`, `sc_platform`); no server-side session store.
- **RBAC** — `services/platformUsers/platformPermissions.ts` (`matrix + permits()`), `platformAccess.ts` (`getPlatformViewer` → site scope).
- **Config** — `lib/config.ts` (env access), `lib/prisma.ts` (client singleton).

8. Technology stack

Layer	Choice
Language	TypeScript 5.5
Framework	Next.js 14.2 (App Router, server components)
UI	React 18.3, Tailwind CSS 3.4 (custom design tokens via CSS vars)
ORM / DB	Prisma 5.22 / PostgreSQL 16
Auth	Azure AD via <code>@azure/msal-node</code> (admin); custom signed-cookie sessions (worker,

Layer	Choice
	platform)
SMS	@azure/communication-sms (ACS) + Twilio stub; mock provider active
Hosting	Azure App Service (Linux, Node 22-lts)
Tooling	ESLint (next config), Prettier (+ tailwind plugin), tsx
Node	local dev on 20.x; App Service runtime Node 22-lts. engines.node >=20.19.0

No test framework is configured (see §25 — this is the largest technical-debt item).

9. Folder structure

```

sitecomply/
├─ app/                                # Next.js App Router (pages + API route handlers)
│   └─ page.tsx                        # Landing (Worker / Platform Login / Admin)
│   └─ privacy/                        # Privacy notice
│   └─ check-in/                       # Worker journey (OTP → site → induction → confirm)
│   └─ admin/                          # Admin area
│       └─ login/                      # Microsoft sign-in page
│           └─ (dashboard)/            # Auth-gated: sites, submissions, on-site,
│               |                     #   platform-users, checklist builder
│           └─ platform/               # Platform Login + dashboard
│               └─ (page)/              # /platform (login chooser), /platform/email, /mobile
│                   └─ dashboard/       # Gated area: dashboard + sections (sites, submissions,
│                       |               #   reports, documents, audits, actions) + layout gate
│                   └─ api/              # Route handlers (see §13)
│                       └─ worker/ admin/ platform/ health/
├─ components/                         # React components
│   └─ ui/                             # Button, TextField, Textarea, ConfirmDialog, Toast
│   └─ layout/ brand/                  # AppShell, Logo
│   └─ checkin/ admin/                # journey + admin widgets
│       └─ platform/                  # PlatformShell, PlatformNav, SectionPreview, icons
├─ services/                           # Business logic (see §10/§12/§15)
│   └─ auth/ admins/ workers/ sites/ submissions/ checklists/ onsite/ dashboard/
│   └─ sms/                           # SmsProvider interface + acs/twilio/mock
│   └─ platformUsers/                 # RBAC + platform access
│       └─ reports/                   # Reports foundation (Phase 0)
├─ lib/                                # cn, config, prisma, session, phone, postcode,
│   |                                #   datetime, cscs, csv
├─ prisma/                             # schema.prisma + migrations/
├─ docs/                               # RBAC.md, REPORTS.md
├─ scripts/                            # local-db.sh (userland Postgres helper)
├─ public/                             # logo/favicon assets
└─ DEPLOYMENT.md README.md LOCALISATION.md

```

10. Database schema & Prisma models

PostgreSQL via Prisma. All timestamps UTC; British formatting at the edge (`lib/datetime.ts`). IDs are `cuid()`.
Source of truth: `prisma/schema.prisma`.

Enums

- `CscsCardType` — GREEN_LABOURER, RED_TRAINEE, BLUE_SKILLED, GOLD_SUPERVISORY, BLACK_MANAGER, WHITE_PROFESSIONAL
- `SiteStatus` — ACTIVE, ARCHIVED
- `ChecklistItemType` — ACKNOWLEDGEMENT, YES_NO, PPE_CONFIRM
- `SubmissionStatus` — COMPLIANT, INCOMPLETE
- `AdminRole` — OWNER, ADMIN, VIEWER
- `PlatformRole` — DIRECTOR, PROJECT_MANAGER, CLIENT, AUDITOR, ENGINEER, HS_CONSULTANT, PRINCIPAL_CONTRACTOR, SITE_MANAGER
- `PlatformUserStatus` — PENDING, ACTIVE, DISABLED

Models (key fields)

- **Worker** — `id`, `fullName`, `company`, `mobile` (unique, E.164), `cscsCardNumber?`, `cscsCardType?`, `cscsExpiry?`, `createdAt`, `updatedAt` → `submissions[]`
- **JobSite** — `id`, `name`, `addressLine1`, `addressLine2?`, `town`, `postcode`, `jobReference`, `status`, `inductionContent`, `fireAssemblyPoint?`, `firstAiderName?`, `firstAiderNumber?`, `createdByAdminId` → `checklists[]`, `submissions[]`, `platformUsers[]`
- **ComplianceChecklist** — `id`, `jobSiteId`, `title`, `version`, `createdAt` → `items[]`; unique (`jobSiteId`, `version`) (editing a checklist creates a new version)
- **ChecklistItem** — `id`, `checklistId`, `label`, `helpText?`, `type`, `required`, `order`
- **Submission** — `id`, `workerId`, `jobSiteId`, `checklistVersion`, `answers(Json)`, `ppeConfirmed`, `rulesAcknowledged`, `safeWorkingAgreed`, `gdprConsent`, `status`, `checkedInAt`, `checkedOutAt?`
- **Admin** — `id`, `azureObjectId` (unique), `email` (unique), `displayName`, `role(AdminRole)`, `createdAt`, `lastLoginAt?` → `createdSites[]`
- **OtpChallenge** — `id`, `mobile`, `codeHash` (HMAC – code never stored), `expiresAt`, `attempts`, `consumedAt?`, `createdAt`
- **PlatformUser** — `id`, `name`, `company`, `email` (unique), `mobile?`, `role(PlatformRole)`, `status(PlatformUserStatus)`, `createdAt`, `updatedAt` → `assignedSites[]` (m2m JobSite), `reportExports[]`
- **ReportExportLog** — `id`, `platformUserId`, `role(PlatformRole)`, `reportType`, `format`, `siteIds(Json)`, `dateFrom?`, `dateTo?`, `rowCount`, `createdAt` (export audit log — Admin-only viewing)

```
Worker 1-* Submission *-1 JobSite 1-* ComplianceChecklist 1-* ChecklistItem
Admin 1-* JobSite
PlatformUser *-* JobSite (assignedSites) PlatformUser 1-* ReportExportLog
OtpChallenge (standalone, keyed by mobile)
```

11. Authentication flows

Three independent flows, each a signed cookie from `lib/session.ts` (`signSession/verifySession`, HMAC-SHA256 keyed by `SESSION_SECRET`).

11a. Worker SMS OTP (`sc_worker`, 2h)

```
/check-in → enter UK mobile → POST /api/worker/otp/request
  → services/auth/otpService: normalise to E.164, rate-limit, store OtpChallenge
  (HMAC of code only), send via SmsProvider (MOCK: code logged to console)
→ enter code → POST /api/worker/otp/verify → verify HMAC + attempts/expiry
  → set sc_worker session → identity → site select → induction wizard
  → POST /api/worker/submission (compliant + checked in) → confirmation
  → POST /api/worker/checkout on leaving
```

Codes are never stored (only an HMAC hash); short TTL (`OTP_TTL_SECONDS`), attempt cap, per-mobile rate limiting.

11b. Admin — Azure AD / Entra ID SSO (`sc_admin`, 8h)

```
/admin/login → "Sign in with Microsoft" → GET /api/admin/auth/login
  → services/auth/adminAuth (MSAL confidential client) builds authorize URL
  (state cookie for CSRF) → Microsoft login
→ GET /api/admin/auth/callback → exchange code → id token → upsert Admin
  (by azureObjectId) → set sc_admin session → /admin
```

Config: `AZURE_AD_TENANT_ID/CLIENT_ID/CLIENT_SECRET/REDIRECT_URI`. Live in production (Entra app registered, redirect `https://app.sitecomply.co.uk/api/admin/auth/callback`). Locally, if Azure AD is unconfigured **and** not production, a dev sign-in creates a dev admin (never in production).

11c. Platform Login (`sc_platform`, 8h)

```
/platform → Login with Email OR Mobile → enter identifier
  → POST /api/platform/auth/start → look up PlatformUser by email/mobile
  → ACTIVE: return dev code "123456"; PENDING: "awaiting approval";
  DISABLED: "access revoked"; unknown: not found
→ enter code → POST /api/platform/auth/verify → re-check ACTIVE + code
  → set sc_platform session (stores userId only) → /platform/dashboard
```

 **The platform verification code is a hardcoded development value 123456** (in `app/api/platform/auth/{start,verify}` and the two login pages). No real email/SMS is sent. **This must be replaced with a real OTP before any real-world platform launch** (§25/§26).

12. RBAC — roles & permissions

Full design: `docs/RBAC.md`. Implementation: `services/platformUsers/`.

- **Site scope** (`platformAccess.ts` → `getPlatformViewer`): **Director = all sites**; every other role = **assigned**

sites only (`viewer.siteIds`). Enforced on every read/write; non-assigned sites are invisible. Status re-checked per request (disable/removal takes effect immediately).

- **Permissions** (`platformPermissions.ts`): the approved matrix

`PLATFORM_PERMISSIONS[role].modules[module] = verbs[], verbs view|create|edit|export`. All 8 roles enforced via `permits()`.

Matrix (verbs per module):

Module	Director	Project Mgr	Site Mgr	Client	Auditor	Engineer	H&S	Principal Contractor
Dashboard	V	V	V	V	V	V	V	V
Sites	VCEX	VCEX	VE	V	V	V	VE	VCEX
Check-ins	VX	VX	VX	V	VX	V	VX	VX
Documents	VCEX	VCEX	VCEX	V	VX	VCE	VCEX	VCEX
Audits	VX	VCEX	VX	V	VCEX	V	VCEX	VCEX
Reports	VCEX	VCEX	VX	V	VCX	V	VCX	VCX
Actions	VCEX	VCEX	VCE	V	VC	VCE	VCE	VCEX
Platform Users	—	—	—	—	—	—	—	— (Admin-only)

Key rules: **Director = all sites**; **Client = read-only** (no create/edit/export; "Read-only" badge); **Export =** Director/PM/Site Mgr/Auditor/H&S/PC (Client & Engineer never export worker/personal data); **CSCS detail export** further restricted to Director/PM/Site Mgr/H&S; **Platform Users** managed only by Admins (Microsoft SSO), never by platform roles.

```
// services/platformUsers/platformPermissions.ts (core check)
export function permits(role, module, verb) {
  if (!isRbacEnforced(role)) return true; // (all 8 roles now enforced)
  return can(role, module, verb); // matrix lookup
}
```

Applied at: navigation (`PlatformNav`), page access (`assertModuleView`), action buttons (export buttons), and API routes (`/api/platform/sites/export`, `/submissions/export` return 403 when not permitted, scoped to `viewer.siteIds`).

13. API endpoints

All are Next.js route handlers (`runtime = 'nodejs'`, `dynamic = 'force-dynamic'`).

Health

- `GET /api/health` — liveness (no DB), returns `{status:"ok"}`.

Worker (session `sc_worker`)

- `POST /api/worker/otp/request` — request SMS OTP
- `POST /api/worker/otp/verify` — verify OTP → session
- `POST /api/worker/profile` — save/confirm worker identity
- `POST /api/worker/submission` — record induction + check-in
- `POST /api/worker/checkout` — check out

Admin (session `sc_admin`; Azure AD)

- `GET /api/admin/auth/login` · `GET /api/admin/auth/callback` · `GET /api/admin/auth/logout`
- `POST /api/admin/sites` · `PUT|DELETE /api/admin/sites/[id]` · `POST /api/admin/sites/[id]/status` · `PUT /api/admin/sites/[id]/checklist`
- `POST /api/admin/platform-users` · `PUT|DELETE /api/admin/platform-users/[id]` · `POST /api/admin/platform-users/[id]/status`
- `GET /api/admin/submissions/export` — CSV of check-ins
- `POST /api/admin/workers/[id]/erase` — UK-GDPR personal-data erasure

Platform (session `sc_platform`)

- `POST /api/platform/auth/start` — validate account + status (dev code)
- `POST /api/platform/auth/verify` — verify code → session
- `GET /api/platform/auth/logout`
- `GET /api/platform/sites/export` — CSV of accessible sites (RBAC + scope gated)
- `GET /api/platform/submissions/export` — CSV of check-ins (RBAC + scope gated)

(Reports export endpoints `/api/platform/reports/*` arrive in Phase 1.)

14. Frontend routes & pages

Public / worker

- `/` landing (Worker / Platform Login / Admin) · `/privacy`
- `/check-in` (OTP) · `/check-in/details` · `/check-in/site` · `/check-in/site/[siteId]` · `/check-in/site/[siteId]/induction` · `/check-in/confirmation/[submissionId]`

Admin (gated by (dashboard) layout)

- `/admin/login`
- `/admin (dashboard) · /admin/sites · /admin/sites/new · /admin/sites/[id] · /admin/sites/[id]/checklist · /admin/on-site · /admin/submissions · /admin/submissions/[id] · /admin/platform-users · /admin/platform-users/new · /admin/platform-users/[id]`

Platform

- `/platform (login chooser) · /platform/email · /platform/mobile`
- `/platform/dashboard` and sections: `/sites /submissions /reports /documents /audits /actions` (gated by `app/platform/dashboard/layout.tsx`)

15. Reporting module (in progress)

Design + phased plan: `docs/REPORTS.md`. **Phase 0 shipped** (c13eb6e):

- **Registry** `services/reports/reportRegistry.ts` — 7 report types (Site Attendance, Compliance, On-Site Occupancy, Workforce & Company, CSCS/Competency, Expiring CSCS Cards, Organisation Overview [Director-only]) with metadata (`directorOnly`, `personalData`, `clientAggregateOnly`, `exportRoles`).
- **Access helpers** `services/reports/reportAccess.ts` — `canRunReport`, `canExportReport` (incl. CSCS restriction), `isAggregateOnly` (Client), `getVisibleReports`, `resolveReportScope` (requested n accessible).
- **Audit log** `services/reports/reportExportLog.ts` + `ReportExportLog` model — `logReportExport` (called by every export), `listReportExportLogs` (Admin-only view — later phase).
- **CSV util** `lib/csv.ts` (now shared by the Sites/Check-ins export routes).
- **Landing** `/platform/dashboard/reports` — catalogue filtered by role & scope (cards currently "Coming soon", not yet linked).

Locked decisions: Client = aggregate-only (incl. CSCS); CSCS detail export = Director/PM/Site Mgr/H&S; export audit log = **Admin-only**; v1 = CSV first, **no** saved/scheduled/email/custom-builder; added **Expiring CSCS Cards (30/60/90-day)**.

Next (Phase 1): build Site Attendance + Compliance report pages +

`/api/platform/reports/{attendance,compliance}/export` (scoped, gated, `logReportExport` wired, Client aggregate-only).

16. Document management module

Not implemented. `/platform/dashboard/documents` is a preview stub (`SectionPreview`). No Document model, storage, or API. RBAC verbs exist in the matrix; needs a model, blob storage (e.g. Azure Blob), upload/download UI, and per-module export gating.

17. Audit module

Not implemented. `/platform/dashboard/audits` is a preview stub. Matrix includes audit create/edit + **sign-off** (Auditor/H&S/PC). Needs an `Audit/AuditFinding` model, forms, sign-off workflow, and audit reports (deferred in

Reports plan).

18. Actions module

Not implemented. /platform/dashboard/actions is a preview stub. Matrix includes create/edit; needs an Action/task model, assignment, and completion workflow.

19. Site management module

- Admin:** full CRUD (/admin/sites), versioned induction **checklist builder** (/admin/sites/[id]/checklist), archive/restore (confirm dialog), permanent delete (guarded when workers are checked in). Services: services/sites/*, services/checklists/*.
- Platform:** read-only, **site-scoped** list (/platform/dashboard/sites) + gated **Export CSV** (Director/PM/PC). Non-assigned sites hidden.

20. Azure services used

Service	Resource	Status
App Service (Linux, Node 22-lts)	sitecomply-web (plan sitecomply-plan, B1)	✔ Live
PostgreSQL Flexible Server (PG16, B1ms)	sitecomply-pg, DB sitecomply	✔ Live
Azure AD / Entra ID (admin SSO)	app registration (home tenant 6a8a12c9...)	✔ Live
Custom domain + managed TLS	app.sitecomply.co.uk (SNI cert, auto-renew)	✔ Live
Azure Communication Services (SMS)	—	✖ Not registered (Microsoft.Communication provider not enabled); worker SMS is MOCK
Key Vault	—	✖ Not used (secrets in App Service settings)
Application Insights	—	✖ Not enabled (Microsoft.Insights not registered)

Resource group: **rgSiteComply**. Azure access is **Contributor on this RG only** (no subscription-scope rights; Graph/Directory writes denied — the Entra app was created out-of-band). Subscription: *Microsoft Azure Sponsorship*.

21. Environment variables

From `.env.example` (production values are Azure App Service application settings):

Var	Purpose
<code>APP_BASE_URL</code>	Public base URL (SSO redirects, links). Prod: <code>https://app.sitecomply.co.uk</code>
<code>NODE_ENV</code>	<code>production</code> in prod (disables dev-only fallbacks)
<code>DATABASE_URL</code>	Postgres connection (<code>sslmode=require</code> in prod)
<code>AZURE_AD_TENANT_ID</code> / <code>CLIENT_ID</code> / <code>CLIENT_SECRET</code> / <code>REDIRECT_URI</code>	Admin Microsoft SSO
<code>SESSION_SECRET</code>	HMAC key for all signed-cookie sessions (≥ 16 chars; required in prod)
<code>SMS_PROVIDER</code>	<code>acs</code> <code>twilio</code> <code>mock</code> (prod currently mock)
<code>ACS_CONNECTION_STRING</code> / <code>ACS_SMS_SENDER</code>	Azure Communication Services SMS
<code>TWILIO_ACCOUNT_SID</code> / <code>AUTH_TOKEN</code> / <code>SMS_FROM</code>	Twilio (stub alternative)
<code>OTP_TTL_SECONDS</code> / <code>OTP_LENGTH</code>	Worker OTP behaviour

The platform verification code `123456` is **hardcoded in source**, not an env var.

22. Deployment architecture

- **Host:** Azure App Service (Linux). **Startup command:** `node node_modules/next/dist/bin/next start` (invoking the Next binary via `node` avoids stripped exec-bit issues in zipped `node_modules`).
- **Build/deploy:** Oryx build-on-deploy fails for this app (`@/*` alias resolution), so deploy a **prebuilt zip**: build locally (`npm run build`), zip `.next` + `node_modules` + `public` + `prisma` + `package*.json` + `next.config.js`, `az webapp deploy --type zip` with `SCM_DO_BUILD_DURING_DEPLOYMENT=false`.
- **Gotchas (all real, documented):** `az webapp deploy` often returns **504 but completes server-side** — verify via server `.next/BUILD_ID` (Kudu VFS). A plain restart is unreliable — do a **hard `az webapp stop` + `start`** to cycle onto new code. The new container can take **~1–2 min** to warm up. Bumped `WEBSITES_CONTAINER_START_TIME_LIMIT=600`.

- **Migrations:** run as a **release step**, not in startup. Prod Postgres firewall is Azure-services-only, so from a dev box add a temporary firewall rule for your IP: `az postgres flexible-server firewall-rule create --resource-group rgSiteComply --server-name sitecomply-pg --name <rule> --start-ip-address <IP> --end-ip-address <IP>` (note: `--server-name` for the server, `--name` for the RULE), then `DATABASE_URL="$PROD" npx prisma migrate deploy`, then delete the rule. **Migrate the DB before deploying code that needs the new tables.**
 - **No CI/CD** — deploys are manual (DEPLOYMENT.md sketches a GitHub Actions pipeline that is not implemented).
-

23. Migration status

prisma/migrations/ (5):

1. 20260625060232_init
2. 20260625061221_add_otp_challenge
3. 20260701052215_add_platform_users
4. 20260701054157_add_site_manager_role
5. 20260702002527_add_report_export_log ← **committed, NOT yet applied to production**

Production DB has 1–4 applied. Migration 5 (Reports Phase 0) is in the repo but must be run on Azure Postgres before/with deploying c13eb6e.

24. Git commit history summary

29 commits on `feature/archived-badge-style` (the deployed branch). Arc:

- 8f27399 ... b20b6af — initial build, branding, working admin dashboard, toast/check-out, dashboard stat cards.
 - Merges b260341/f95c041/586798f — delete-site, archive-confirm dialog, check-in/out polish.
 - 74b8c61 ... 81ab114 — favicon/logo; 17f6996 dashboard cleanup; c7223e1 admin-auth redirect fix.
 - 9f9bd65 Platform Users management → f86c34c Site Manager role → 9def4b1 RBAC design+matrix.
 - 34125a8 ... ec1f23d ... d1d1388 — Platform Login journey + Dashboard (UI) + dev code + polish.
 - b6f14f2 Platform Login auth + Assigned-Sites enforcement + RBAC → b2a9f46 RBAC Phase 2 (all roles) **[deployed]**.
 - c13eb6e Reports Phase 0 **[latest; not deployed]**.
-

25. Known issues, technical debt, TODO

-  **Platform login uses a hardcoded dev code 123456** — any ACTIVE platform user can sign in with it. Must be replaced with real email/SMS OTP before real platform use.
-  **No automated tests** — no test runner/tests exist; all verification has been manual. High regression risk

as the app grows.

- **Worker SMS is mock** — `Microsoft.Communication` provider not registered; codes only logged to console. Real SMS blocked on ACS registration (subscription-scope admin action) or Twilio credentials.
- **Documents / Audits / Actions** are UI stubs only — no models, storage, or logic.
- **Reports Phase 0 not deployed** — commit `c13eb6e` + migration 5 pending on prod.
- **Secrets in App Service settings, not Key Vault**. No secret rotation automation.
- **No observability** — App Insights not enabled.
- **Data-residency ambiguity** — `DEPLOYMENT.md` assumes `uksouth`; resource group is `rgSiteComply` (documented as `northeurope`). Verify UK-GDPR data residency of the App Service + Postgres region.
- **Manual, fragile deploy** (prebuilt zip; 504s; hard stop/start). No CI/CD.
- **Branch strategy** — production deploys from `feature/archived-badge-style`, not `main`; `main` lags. Consolidate.
- **Test data in prod** — verification users (`director@test.com`, `pm@test.com`, `client@test.com`, `pending@test.com`) and `Test Site A/B` were created via admin to verify RBAC and **may still exist** — remove via Admin → Platform Users (Remove) and Job sites (Delete).
- **Single B1 instance** — sessions are stateless cookies (scale-safe), but no redundancy/scale-out configured.

26. Risks

Risk	Impact	Mitigation
Dev auth code live in production platform login	Unauthorised platform access	Replace with real OTP before launch; treat platform as preview until then
No automated tests	Regressions ship unnoticed	Add unit (permissions/scope/date) + integration (export gating) + e2e per role
UK-GDPR data residency unverified	Compliance exposure	Confirm/relocate to UK region; document DPIA
Secrets outside Key Vault	Secret leakage / rotation gaps	Move to Key Vault references; rotate AAD secret + <code>SESSION_SECRET</code>
Manual deploy	Human error, downtime	Implement the GitHub Actions pipeline; migrate-then-deploy discipline
Worker SMS mock	Cannot onboard real workers at scale	Register ACS / configure Twilio

27. Deployment readiness

- **Worker check-in + Admin:** production-ready and **live** (real Microsoft SSO; worker SMS is the one gap — mock only).
 - **Platform Login + RBAC:** **live and verified in production**, but gated behind a **dev verification code** — treat as preview/UAT, not real external access, until real OTP lands.
 - **Reports Phase 0:** committed, **not deployed** (needs migration 5 + a deploy).
 - **Before a real platform launch:** real platform OTP, ACS/Twilio SMS, automated tests + CI/CD, Key Vault, App Insights, data-residency confirmation.
-

28. Recommended next steps (in order)

1. **Deploy Reports Phase 0** — run migration `add_report_export_log` on prod Postgres, then deploy `c13eb6e` (prebuilt zip → hard stop/start → verify).
 2. **Build Reports Phase 1** — Site Attendance + Compliance reports; scoped CSV export routes with `logReportExport`; Client aggregate-only. (Foundation already in place: `reportRegistry`, `reportAccess`, `lib/csv`, `ReportExportLog`.)
 3. **Continue Reports Phases 2–5** per `docs/REPORTS.md` (Occupancy/Workforce → CSCS + Expiring CSCS → Org Overview → hardening + Admin-only export-log view + tests).
 4. **Replace platform dev code with real OTP** — reuse the worker OTP pattern (`OtpChallenge`, HMAC) for platform email/SMS; register **Azure Communication Services** (needs subscription-scope provider registration) or an email provider.
 5. **Introduce automated testing + CI/CD** — start with the RBAC/scope helpers and export gating; wire GitHub Actions (build → migrate → deploy).
 6. **Harden infra** — Key Vault for secrets, Application Insights, confirm UK data residency, consolidate to a `main` deploy branch.
 7. **Clean up prod test data** — remove the RBAC verification users and Test Sites A/B.
 8. **Then** build the Documents, Audits, and Actions modules (models + storage + UI), gating each with the existing `permits()` + site-scope layer.
-

Quick start for a new developer

```
# Toolchain is userland (no sudo); wrappers are on PATH via ~/.local/bin
cd ~/sitecomply
scripts/local-db.sh start          # local Postgres 16
cp .env.example .env              # set DATABASE_URL, SESSION_SECRET (>=16 chars)
npm install
npx prisma migrate dev            # apply migrations locally
npm run dev                       # http://localhost:3000
# Admin dev sign-in works only under `next dev` (not `next start`) when AZURE_AD_* unset.
# Worker OTP + platform code: dev mode shows the code on screen (platform code = 123456).
npm run typecheck && npm run lint # gates used throughout (there is no test suite yet)
```

End of handover. Latest commit at time of writing: `c13eb6e`.